E-GraphSAGE: A Graph Neural Network based Intrusion Detection System for IoT

Wai Weng Lo*, Siamak Layeghy†, Mohanad Sarhan‡, Marcus Gallagher§, and Marius Portmann¶
School of Information Technology and Electrical Engineering
The University of Queensland, Brisbane, Australia
Email: *w.w.lo@uq.net.au, †siamak.layeghy@uq.net.au, ‡m.sarhan@uq.net.au, §marcusg@itee.uq.edu.au, ¶marius@itee.uq.edu.au

NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium | 978-1-6654-0601-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/NOMS54207.2022.9789878

***Abstract*—This paper presents a new Network Intrusion Detection System (NIDS) based on Graph Neural Networks (GNNs). GNNs are a relatively new sub-field of deep neural networks, which can leverage the inherent structure of graph-based data. Training and evaluation data for NIDSs are typically represented as flow records, which can naturally be represented in a graph format. In this paper, we propose E-GraphSAGE, a GNN approach that allows capturing both the edge features of a graph as well as the topological information for network intrusion detection in IoT networks. To the best of our knowledge, our proposal is the first successful, practical, and extensively evaluated approach of applying GNNs on the problem of network intrusion detection for IoT using flow-based data. Our extensive experimental evaluation on four recent NIDS benchmark datasets shows that our approach outperforms the state-of-the-art in terms of key classification metrics, which demonstrates the potential of GNNs in network intrusion detection, and provides motivation for further research.**

***Index Terms*—Graph Neural Networks, Network Intrusion Detection System, Internet of Things**

I. Introduction

IoT network attacks have significantly increased over the last few years, both in terms of frequency and level of sophistication. IoT networks consist of many interconnected devices, including cameras, temperature sensors, smart TV, wireless printer, and other edge devices that require network connectivity [1]. Cybercriminals can compromise and exploit IoT networks to perform malicious activities such as IoT ransomware, Botnet DDoS attacks. NIDSs, which are placed at strategic points within the IoT network to monitor traffic, are an important tool for the detection and mitigation of network-based cyber attacks. There are two main types of NIDS, signature-based and intrusion detection-based systems. A signature-based NIDS relies on a pre-installed set of attack signatures, which are compared and pattern-matched with the monitored network traffic in order to detect attacks. As a result, this type of NIDS can effectively detect known attacks with a relatively low false alarm rate, but they are significantly less effective in detecting new attacks or variants of existing attacks. In contrast, intrusion detection-based systems have the potential to detect intrusions by detecting deviations from the regular traffic patterns in the network. Over the last few years, there has been great progress in the application of new Machine Learning (ML) models and techniques, in particular deep learning-based approaches, for the development of new NIDS solutions.

In this paper, we are exploring the use of GNNs, a relatively new sub-field of deep neural networks for IoT network intrusion detection. GNNs are tailored to applications with graph-structured data, such as social sciences, chemistry, and telecommunications, and are able to leverage the inherent structure of the graph data by building relational inductive biases into the deep learning architecture. This provides the ability to learn, reason, and generalise from the graph data, inspired by the concept of message propagation [2].

Network intrusion detection is typically performed on flow-based network data such as NetFlow [3], where flows are identified by the communication endpoints (IP address, L4 port number, L4 protocol), and annotated via a set of flow fields which provide details on the flows, such as the number of packets, number of bytes, flow duration, etc. This flow data can naturally be represented in a graph format, where the flow endpoints are mapped to graph nodes, and network traffic flows are mapped to the graph edges. Both the topological information as well as the information contained in edge features are crucial for the classification of network traffic and the detection of attack flows.

Key recent works on ML-based NIDS, such as [4][5][6][7][8], consider flow data records independently, without considering the interrelation between them, and hence the traffic pattern more globally. Consequently, these methods are limited in their ability to detect sophisticated IoT network attacks, such as botnet attacks [9], distributed port scans [10], or DNS Amplification attacks [11], where a more global view of the network and traffic flow is required.

We believe that traffic patterns at a more global level, and hence flow interrelationships should be considered for the detection of distributed attacks in IoT, providing the motivation for the work presented in this paper. We propose E-GraphSAGE, a GNN model that aims to overcome the current limitations of NIDSs for IoT. E-GraphSAGE enables the collection of information on flow-based features and topological patterns to account for the interconnected patterns

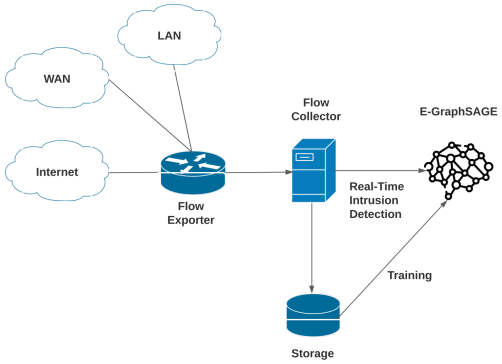


Fig. 1: Overall deployment architecture

of network flows.Consequently, E-GraphSAGE supports the process of edge classification, and hence the detection of malicious network flows, as illustrated in Figure 1.

We demonstrate how the E-GraphSAGE algorithm can be utilized to build a reliable NIDS, and provide an extensive experimental evaluation of the proposed system on four recent NIDS benchmark datasets. Our results show that the E-GraphSAGE-based NIDS outperformed the state-of-the-art in regards to key classification metrics in all four considered benchmark datasets. To the best of our knowledge, our approach is the first successful, practical, and extensively evaluated approach of applying GNNs on the problem of network intrusion detection for IoT using flow-based data.

In summary, the key contributions of this paper are twofold:

- We have proposed and implemented E-GraphSAGE, a GNN-based NIDS, which allows the incorporation of edge features and topological patterns for IoT network intrusion detection.
- We applied E-GraphSAGE on four benchmark IoT NIDS datasets for network intrusion detection, where the results demonstrated its potential via extensive experimental evaluation.

The rest of the paper is organized as follows. Section II discusses key related works, and Section III provides the relevant background on GNNs and GraphSAGE. Our proposed E-GraphSAGE algorithm and the corresponding NIDS are introduced in Section IV. Experimental evaluation results are presented in Section VI, and Section VII concludes the paper.

II. Related Work

Haitao et al.[4] designed a multimodal sequential NIDS with a hierarchical progressive network to capture different levels of network data features. The approach is based on a combination of a deep autoencoder and LSTM architecture, in order to integrate the structural information within the temporal information shared between similar network connections. The system is evaluated on 3 datasets including the UNSW-NB15 dataset, where it achieved an accuracy of 96.8% and 86.2% in binary and multiclass classification experiments respectively.

In [5], an attack mitigation framework for IoT networks is proposed using a hybrid of signature- and intrusion-based detection systems. The signature-based system uses a database of black-listed sources. The intrusion-based module adopts an extreme gradient boosting (XGBoost) algorithm. The XGBoost classifier achieved a binary classification accuracy of 99.99% and a false alarm rate of 0.05% on the BoT-IoT dataset, and 99.96% multiclass classification F1-Score of 97%.

Sarhan et al. [6] converted the UNSW-NB15, BoT-IoT, and ToN-IoT datasets from their proprietary formats and feature set into a common Netflow-based format, with eight Netflow features as a common feature set. The corresponding new Netflow-based variants of the datasets, i.e. NF-UNSW-NB15, NF-BoT-IoT, and NF-ToN-IoT, have been made publicly available. The authors evaluated an Extra Tree ensemble classifier across these three datasets and have reported an F1-Score of 0.85, 0.97, and 1.00, respectively for binary classification. For the multi-class case, the corresponding F1-Scores are 0.98, 0.77 and 0.60.

In [7], the authors focused on securing Internet of medical things' networks (IoMT) using a two-level intrusion detection model. The first level used a decision tree (DT), naive Bayes (NB), and random forest (RF) as first-level individual learners. In the next level, an XGBoost classifier was used to identify normal and attack instances. The ensemble model achieved a binary classification accuracy of 96.35% and an F1-Score of 0.95 on the ToN-IoT dataset.

Churcher et al. [8] applied several machine learning algorithms such as k-nearest neighbour (KNN), decision tree (DT), support vector machine (SVM), naive Bayes (NB), random forest (RF), artificial neural network (ANN), and logistic regression (LR) for network intrusion detection. The authors evaluated different classifier performances on BoT-IoT datasets and reported that the KNN classifier achieved the highest multiclass classification performance with an F1-Score of 0.99 and classification accuracy of 99.00%.

Xiao et al. [12] proposed a graph embedding approach to perform intrusion detection on network flows. The authors first converted the network flows into a first-order and second-order graph. The first-order graph learns the latent features from the perspective of a single host by using its IP address and port number. The second-order graph aims to learn the latent features from a global perspective by using source IP addresses, source ports, destination IP addresses, as well as destination ports. The extracted graph embeddings and the raw features are then used to train a Random Forest classifier to detect network attacks.

However, a significant limitation of this approach is its use of a traditional transductive graph embedding method [13], which limits its ability to classify samples with graph nodes, e.g. IP addresses and port numbers, which were not seen during the training phase. This makes the approach unsuitable for most practical NIDS application scenarios. In contrast, the E-GraphSAGE approach presented in this paper uses an inductive learning approach, which does not suffer from this limitation.

Zhou et al.[14] proposed using a graph convolutional neural network (GCN) to perform P2P botnet node detection. The authors first generated botnet traffic by creating botnet connections mixed with different real large-scale network traffic flows. Then, they applied the GCN for botnet node classification.The generated graph does not include any flow or node features and the approach only considers the topological information of the network connectivity graph for P2P botnet node classification, rather than flow classification. This approach limits the detection of botnet attacks, and does not focus on other network attacks, such as XSS and ransomware. In addition, it does not leverage all the information provided in network flow data, i.e., the network flow features.

While some existing graph representation learning methods have already considered edge features, none of them can be directly applied to network intrusion detection. Methods such as [15] [16] consider edge features, but only for the purpose of improving node representation for better performance, and not edge classification, which is the aim in NIDSs.

In contrast to related studies, our approach leverages edge features that can be extracted from network flow data, and is critical for the detection of individual attack flows via its edge embedding approach. NIDSs based on traditional (non-graph-based) machine learning algorithms currently have not fully leveraged the structural and topological information inherent in network flow data for complex network attack detection, such as a botnet attack, which is one of the key strengths of GNNs. A key feature and novel contribution of E-GraphSAGE proposed in this paper, is its ability to leverage both, topological information and edge features in network flow data. Furthermore, related studies that apply a ML approach for NIDS typically use either, one, two, or a maximum of three NIDS benchmark datasets to evaluate the proposed systems. For the evaluation of the proposed GNN-based approach, we used four different benchmark datasets, which provide a higher degree of confidence in the robustness of the method and the ability to generalize to different types of network scenarios.

III. Background

A. Graph Neural Networks (GNN)

GNN is one of the most recent and fastest growing subareas in machine learning. Its power and potential lies in its ability to leverage the inherent graph structure of a lot of data encountered in real world application domains, such as social media networks, biology, telecommunications, etc. The graph format captures the structural information by modelling a set of objects and their relationships. The objects are represented by graph nodes and their relationships by graph edges. In the case of computer networks, individual hosts (IP addresses) are modelled as graph nodes, and the communication between the hosts, i.e., the network flows, are modelled as graph edges.

The key motivation of using a GNN for NIDSs is the ability to easily and directly exploit the rich structural information in the network flow data, which can be directly encoded in a graph format. While some traditional ML-based approaches attempt to work with graph data, this is typically quite cumbersome and relies on hand-engineered features.

A common task performed by GNNs is to generating node *embeddings* [17], which aims to encode nodes as low-dimensional vectors, while maintaining their key relationships and graph position in the original format. Node embedding is typically a key precursor to apply for downstream tasks such as node classification and node clustering [17]. GNNs have recently received a lot of attention due to their convincing performance and high interpretability of the results through the visualisation of the node *embeddings* [18].

B. GraphSAGE

The *Graph SAmple and aggreGatE (GraphSAGE)* algorithm is one of the most well-known GNNs and was developed by Hamilton et al. [13] . In GraphSAGE, a fixed size sub-set is (uniformly randomly) sampled. This allows limiting the space and time complexity of the algorithm, irrespective of the graph structure (e.g. node degree distribution) and batch size.

The GraphSAGE algorithm operates on a graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes and $\mathcal{E}$ is the set of edges. The node features of a node v are represented as the vector x_v, and the complete set of node feature vectors as $\{x_v, \forall v \in \mathcal{V}\}$.

A key hyperparameter of the GraphSAGE algorithm is the number of *graph convolutional layers* K, which specifies the number of hops via which node information is aggregated at each iteration. Another important aspect of GraphSAGE is the choice of a differentiable aggregator function $AGG_k, \forall k \in \{1, ..., K\}$, to aggregate information from neighbor nodes. The algorithm is performed in both a forward and backward propagation stage, which we now discuss in more detail.

1) Node Embedding: Similar to the convolution operation in convolutional neural networks (CNNs), information relating to the local neighborhood of a node is collected and is used to compute the node embedding. The GraphSAGE algorithm starts by assuming the model has already been trained and the weight matrices and aggregator function parameters are fixed. For each node, the algorithm iteratively aggregates information from the node's neighbours, the node's neighbours-neighbours, and so on.

At each iteration, initially the neighborhood of the node is sampled, and the information from the sampled nodes is aggregated into a single vector. At the k-th layer, the aggregated information $\mathbf{h}^k_{N(v)}$ at a node v, based of the sampled neighborhood $N(v)$, can be expressed as Equation 1 [13]:

$$\mathbf{h}^k_{\mathcal{N}(v)} = \text{AGG}_k\left(\left\{\mathbf{h}^{k-1}_u, \forall u \in \mathcal{N}(v)\right\}\right) \quad (1)$$

Here, $\mathbf{h}^{k-1}_u$ represents the embedding of node u in the previous layer. These embeddings of all nodes u in the neighbourhood of v are aggregated into the embedding of node v at layer k.

This aggregation process is illustrated in Figure 2 (right), the topological and node features in the graph, as collected and aggregated from the k-hop neighborhood of each graph node.

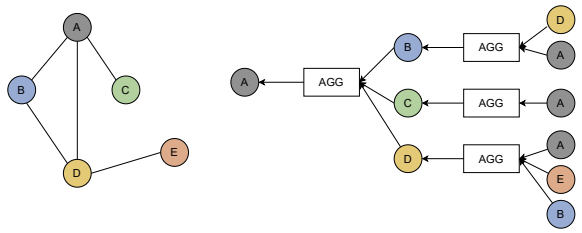


Fig. 2: A given graph (left), and the corresponding GraphSAGE architecture with depth-2 convolutions (right) and full neighborhood sampling.

In the GraphSAGE model, we can use different aggregation methods, including mean, pooling, or different types of neural networks, e.g., Long Short-Term Memory (LSTM) [13].

The aggregated embeddings of the sampled neighborhood $\mathbf{h}^k_{N(v)}$ are then concatenated with the node's embedding from the previous layer $\mathbf{h}^{k-1}_v$. After applying the model's trainable parameters ($\mathbf{W}^k$, the trainable weight matrix) and passing the result through a non-linear activation function σ (e.g. ReLU), the layer k node v embedding is calculated, as shown in Equation 2 [13].

$$\mathbf{h}^k_v = \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}\left(\mathbf{h}^{k-1}_v, \mathbf{h}^k_{\mathcal{N}(v)}\right)\right) \tag{2}$$

The final representation (embedding) of node v is expressed as $\mathbf{z}_v$, which is essentially the embedding of the node at the final layer K, as shown in Equation 3. For the purpose of node classification, $\mathbf{z}_v$ can be passed through a sigmoidal neuron or softmax layer.

$$\mathbf{z}_v = \mathbf{h}^K_v, \qquad \forall v \in \mathcal{V} \tag{3}$$

IV. E-GraphSAGE

Traditional GNNs, e.g. GraphSAGE, have been successfully applied in a wide range of applications. However, these approaches mainly focus on node features for node classification, and are currently unable to consider edge features for edge classification. In contrast, our proposed E-GraphSAGE algorithm allows us to consider the edge (flow) information in the embedding process. This provides the basis for computing the corresponding edge embeddings and enabling edge classification; that is, the classification of network flows into benign and attack flows.

The goal of an NIDS is to detect and identify malicious traffic and flows. This corresponds to the problem of edge classification in our graph representation of flow datasets, where the key information is provided as edge features. Consequently, the current NIDS benchmark network datasets [6] [19] [20] provide network flow information as edge features, rather than node features, which only allows edge (flow) classification. In contrast, most related GNN research to date has focused on the problem of node classification, and the proposed solutions cannot be applied to the edge classification problem in the context of NIDS.

Algorithm 1: E-GraphSAGE edge embedding

input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$;
input edge features $\{\mathbf{e}_{uv}, \forall uv \in \mathcal{E}\}$;
input node features $\mathbf{x}_v = \{1, \ldots, 1\}$;
depth K;
weight matrices $\mathbf{W}^k, \forall k \in \{1, \ldots, K\}$;
non-linearity σ;
differentiable aggregator functions AGG_k ;
output: Edge embeddings $\mathbf{z}_{uv}, \forall uv \in \mathcal{E}$

1 $\mathbf{h}^0_v \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$
2 **for** $k \leftarrow 1$ **to** K **do**
3 **for** $v \in \mathcal{V}$ **do**
4 $\mathbf{h}^k_{\mathcal{N}(v)} \leftarrow \text{AGG}_k\left(\left\{\mathbf{e}^{k-1}_{uv}, \forall u \in \mathcal{N}(v), uv \in \mathcal{E}\right\}\right)$
5 $\mathbf{h}^k_v \leftarrow \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}\left(\mathbf{h}^{k-1}_v, \mathbf{h}^k_{\mathcal{N}(v)}\right)\right)$
6 $\mathbf{z}_v = \mathbf{h}^K_v$
7 **for** $uv \in \mathcal{E}$ **do**
8 $\mathbf{z}^K_{uv} \leftarrow CONCAT(\mathbf{z}^K_u, \mathbf{z}^K_v)$

Our proposed approach (E-GraphSAGE) aims to overcome this limitation and facilitate the capture of edge features and topological information for intrusion detection. This section introduces E-GraphSAGE in two parts. The first part discusses the E-GraphSAGE model and the extensions to the original GraphSAGE algorithm that were made to facilitate edge embedding and edge classification. In the second part, we discuss the application of E-GraphSAGE as an NIDS.

A. E-GraphSAGE Model

1) Edge Embedding: The message passing function in the original GraphSAGE algorithm only considers node features, and edge features are not taken into consideration [13]. To include the edge features, it is required to sample and aggregate the edge information of the graph. In addition, the final output of the algorithm needs to provide an edge embedding rather than the node embedding provided by the original algorithm. Our proposed E-GraphSAGE algorithm with these modifications is shown in Algorithm 1.

Key differences to the original GraphSAGE algorithm [13] are in regards to the algorithm input, the message passing/aggregator function, and the output.

The input includes edge features $\{\mathbf{e}_{uv}, \forall uv \in \mathcal{E}\}$, which are, as mentioned before, missing from the list of inputs in GraphSAGE. Since the flow-based NIDS datasets only consist of flow (edge) features rather than node features. Therefore, we are using only provided edge features. We use the vector $\mathbf{x}_v = \{1, \ldots, 1\}$ to initialise the node features (and initial node embeddings) and the dimension of all one constant vector is the same as the number of edge features, as shown in Line 1 of the algorithm.

In Line 4, instead of using the standard GraphSAGE node aggregator function (Equation 1), our newly proposed neighborhood aggregator function creates the *aggregated embeddings of the sampled neighborhood edges* at the k-th layer, as shown in Equation 4 below.

$$\mathbf{h}^k_{\mathcal{N}(v)} = \text{AGG}_k\left(\left\{\mathbf{e}^{k-1}_{uv}, \forall u \in \mathcal{N}(v), uv \in \mathcal{E}\right\}\right) \tag{4}$$

Here, $\mathbf{e}_{uv}^{k-1}$ are the features of edge uv from $\mathcal{N}(v)$, the sampled neighborhood of node v, at layer k-1. The set $\{\forall u \in \mathcal{N}(v), uv \in \mathcal{E}\}$ represents the sampled edges in the neighborhood $\mathcal{N}(v)$). In Line 5, the node embedding for node v at layer k is calculated as in GraphSAGE (Equation 2), but with the critical difference that in the new algorithm $\mathbf{h}_{\mathcal{N}(v)}^{k}$ is calculated via Equation 4 to include the edge features. Thus, the topological and edge information in the network flow graph is collected and aggregated from the k-hop neighborhood of each network graph node.

The final node embeddings at depth K, $\mathbf{z}_v = \mathbf{h}_v^K$, are assigned in Line 8. Finally, the edge embeddings $\mathbf{z}_{uv}^K$ for each edge uv are calculated as the concatenation of the node embeddings of nodes u and v, as shown in Equation 5 below.

$$\mathbf{z}_{uv}^K = CONCAT\left(\mathbf{z}_u^K, \mathbf{z}_v^K\right), uv \in \mathcal{E} \tag{5}$$

This represents the final output of the forward propagation stage in E-GraphSAGE.

2) Time and Space Complexity: The loops over the k-hop neighbour edges are the most time-consuming of the model as shown in Algorithm 1, Line 4. The upper bound time complexity estimation of E-GraphSAGE can be represented as $O\left(eKnd^2\right)$, where n is the total number of nodes in the network, e is the number of neighbour edges being sampled for each node, K is the number of layers, d is the dimension of the node hidden features. The space complexity of E-GraphSAGE is $O\left(beKd + Kd^2\right)$, where b is the batch size. Since E-GraphSAGE can support a min-batch setting, i.e., a fixed size of neighbour edges are being sampled to improve the training efficiency and reduce memory consumption. Instead of using full neighbourhood sets in Algorithm 1, the per-batch space and time complexity for E-GraphSAGE, which uniformly randomly samples a fixed-size set of edge neighbors, is $O\left(\prod_{i=1}^{K} E_i\right)$, where the sampled edges are $E_i, i \in \{1, \ldots, K\}$.

B. E-GraphSAGE NIDS

Figure 3 shows a high level overview of our proposed NIDS based on E-GraphSAGE. First, the network graph is generated from the network flow data, and is then fed into the supervised training process of the E-GraphSAGE model in the next step. In the last step, the edge embeddings are created, which forms the basis for the classification of edges (network flows) into benign and attack classes. These three steps are explained in the following subsections.

1) Network Graph Construction: Network flows (e.g., NetFlow) are a common format for recording network communication in production networks, and it is also the most common format in the context of NIDS. The flow records usually consist of fields for identifying the source and destination of the communication, and the rest of a record field provides further information on the flow, such as the number of packets and bytes, flow duration, etc. A graph therefore provides a very natural way to model such data.

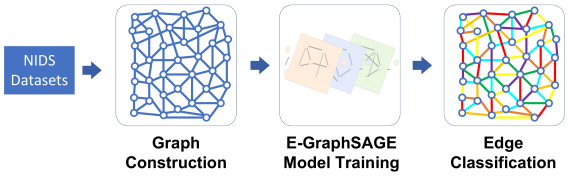


Fig. 3: Proposed E-GraphSAGE-based NIDS architecture.

There are various possibilities for defining the end points of a flow and hence the graph nodes. The method used in this study uses the following 4 flow fields to identify a graph edge: Source IP Address, Source (L4) Port, Destination IP Address, and Destination (L4) Port. The first two fields form a 2-tuple that identifies the source node and the two last fields identify the destination node of the flow. The additional flow fields provide features that are associated with the corresponding graph edge. e.g. , the source node $(172.26.185.48 : 52962)$ exchanges data with destination node $(192.168.1.152 : 80)$ and the corresponding flow can be represented as a network edge.

To construct the network graph from the flow data, we mapped the original source IP addresses to randomly assigned IP addresses in the range from 172.16.0.1 to 172.31.0.1. The reason for this is the fact that in a lot of the NIDS datasets used in this paper, only a small number of IP addresses were used as the source of the attacks. The random mapping avoids the potential problem of the source IP addresses providing an unintentional label for attack traffic.

Since in our graph construction all remaining flow record fields were assigned to the edge, the graph nodes are featureless. In this algorithm, we assign a *one vector*, a vector with all values of one to all nodes. As shown in Algorithm 1, we are using this method in our algorithm, with the dimensionality of the all one constant vector in each node depending on the number of edge features.

2) E-GraphSAGE Training: The neural network model we use in our implementation consists of two E-GraphSAGE layers ($K = 2$), which means that neighbour information is aggregated from a two-hop neighbourhood. For the aggregation function AGG, as shown in Equation 4, we use the mean function which simply finds the element-wise mean of the edge features in the sampled neighborhood. The definition of the mean aggregator function in E-GraphSAGE is provided in Equation 6 below.

$$\mathbf{h}_{\mathcal{N}(v)}^{k} = \sum_{\substack{u \in \mathcal{N}(v), \\ uv \in \mathcal{E}}} \frac{\mathbf{e}_{uv}^{k-1}}{|N(v)|_e} \tag{6}$$

Here, $|N(v)|_e$ represents the number of edges in the sampled neighborhood, and $\mathbf{e}_{uv}^{k-1}$ represent their edge features at layer $k-1$. For our implementation, we chose full neighborhood sampling, which means information from the full set of edges in a node's neighbourhood is aggregated.

In both E-GraphSAGE layers, for the hidden feature sizes of each layer represented in Equation 2, we use a size of 128 hidden units, which is also the dimension of the node embeddings. For the nonlinear transformation (σ in Equation 2), we use the ReLU activation function, and for the purpose of regularisation, we use a dropout mechanism with a rate of 0.2 between the two E-GraphSAGE layers. We chose the cross-entropy loss function, and gradient descent in the backward propagation phase is performed using the Adam optimizer with learning rate of 0.001.

When node embeddings are generated in the last E-GraphSAGE layer, they are converted to the corresponding edge embeddings (Line 10 of Algorithm 1). Since the edge embeddings are created via concatenation of two node embeddings, the size of the edge embeddings is 256 dimensions.

The output edge embeddings pass through a softmax layer, which makes it possible to compare the output of the algorithm to the labels provided by the NIDS datasets and to tune the trainable model parameters in the backward propagation phase.

3) Edge Classification: After the model parameters have been tuned in the training process, the model is ready for evaluation via the classification of unseen test samples. The test flow records are also converted to graphs, passed through the trained E-GraphSAGE layers, from which the edge embeddings are calculated. They are then converted to class probabilities in the final softmax layer and are finally compared with the true class labels in order to compute the classification evaluation performance metrics.

V. Datasets

For the evaluation of the GNN-based NIDS proposed in this paper, we use four publicly available NIDS datasets, which consist of different types of labelled attack flows as well as benign network flows. The first two datasets, consisting of ToN-IoT [20] and BoT-IoT [19], have proprietary formats and feature sets, and have been widely used to evaluate Machine Learning based network intrusion detection systems in IoT. In addition, we also consider two recently created variants of these datasets, where the respective format is translated to NetFlow with a common feature set, i.e. NF-TON-IoT [6] and NF-BoT-IoT [6]. A brief overview of these datasets is provided in the following.

- **BoT-IoT:** This is another recent dataset with a specific on IoT networks, created by Koroniotis et al. [19] in 2019. The authors simulated IoT services such as water stations, by using the Node-red tool [21] and generated the corresponding IoT traffic. The Argus tool was consequently used for feature extraction. This dataset is comprised of 6 types of attacks and a total of 47 features with corresponding class labels. The dataset contains only 477 (0.01%) benign flows and 3,668,045 (99.99%) attack flows, with a total of 3,668,522 flows.
- **ToN-IoT:** This is a relatively new and extensive dataset that was generated in 2019 by Abdullah et al.[20], which includes different types of IoT data, such as operation system logs, telemetry data of IoT/IIoT services, as well as IoT network traffic collected from a medium-scale network at the Cyber Range and IoT Labs at the UNSW Canberra (Australia). In this paper, only the network traffic component of the dataset is used. The Bro-IDS network monitoring tool was used to generate the dataset's 44 network flow features. The dataset consists of 796,380 (3.56%) benign flows and 21,542,641 (96.44%) attack flows, with a total of 22,339,021 flows.
- **NF-ToT-IoT and NF-BoT-IoT:** A lack of a standard format and feature set among the various NIDS datasets has made it very difficult to compare the performance of ML-based network traffic classifiers across different datasets, and to evaluate their ability to generalise to different network scenarios. Sarhan et al. [6] have addressed this problem by providing the NetFlow version of the above mentioned three NIDS datasets. The authors of [6] used the raw packet capture (pcap) files of the original NIDS datasets and converted them to the NetFlow format via the nprobe [22] tool and selected 12 fields to be extracted, resulting in the new variants of the original datasets, i.e. NF-ToT-IoT and NF-BoT-IoT. In NF-ToT-IoT, the total number of network flows is 1,379,274, out of which 1,108,995 (80.4 %) are attack flows and 270,279 (19.6 %) are benign flows. The NF-BoT-IoT dataset has a total number of 600,100 flows, out of which 586,241 (97.69 %) are attack flows and 13,859 (2.31 %) are benign flows.

VI. Experimental Results

To evaluate the performance of the proposed neural network model, the standard metrics listed in Table I are used, where TP, TN, FP and FN represent the number of True Positives, True Negatives, False Positives and False Negatives respectively. First, the results of the binary classification problem, where the aim is to distinguish between attack and benign traffic, are presented. Subsequently, we present the results of the multiclass classification experiments, where the aim was to identify the specific attack type of each flow.

A. Binary Classification Results

The datasets used in this paper all include two sets of labels. The first label determines if a flow record belongs to a benign or attack class, and the second label identifies the attack type. We use the first label set for binary classification, and the second set for multiclass classification.

TABLE I: Evaluation metrics utilised in this study

Metric	Definition
Detection Rate (Recall)	$\frac{TP}{TP+FN}$
Precision	$\frac{TP}{TP+FP}$
F1-Score	$2 \times \frac{Recall \times Precision}{Recall+Precision}$
Accuracy	$\frac{TP+TN}{TP+FP+TN+FN}$
False Alarm Rate (FAR)	$\frac{FP}{FP+TN}$

TABLE II: E-GraphSAGE binary classification results

Dataset	Accuracy	Precision	F1-Score	Recall (DR)	FAR
BoT-IoT	99.99%	1.00	1.00	99.99%	0.00 %
NF-BoT-IoT	93.57%	1.00	0.97	93.43%	0.38 %
ToN-IoT	97.87%	1.00	0.99	97.86%	1.92%
NF-ToN-IoT	99.69%	1.00	1.00	99.85%	0.15%

We use the full dataset, except for the ToN-IoT dataset, where we considered a randomly sampled subset with 10% of the original size, due to the very large size of the original dataset. In regards to training and evaluation data split, 70% of the flow records of each datasets were selected for training and 30% were reserved for testing and evaluation.

As mentioned above, the experimental evaluation was based on three original datasets with proprietary flow formats and feature sets, as well as their corresponding NetFlow versions. As shown in [6], even though the original dataset and its NetFlow counterpart represent essentially the same network events, the performance of a classifier for the two datasets versions can vary significantly, due to the different feature sets. Table II presents the detailed results of our E-GraphSAGE classifier for the binary classification case, showing Accuracy, Precision , F1-Score, Recall and FAR, for the considered four benchmark datasets. As can be seen in the table, the E-GraphSAGE classifier performs very well across all different performance metrics. Since the considered datasets are generally highly imbalanced, the F1-Score is a more relevant performance metric. We use the F1-Score to compare our classifier with the state-of-the-art, i.e., the best classification results reported in the literature for each of the four NIDS datasets.

Table III shows the corresponding results of our E-GraphSAGE classifier compared to the best performing related works for each datasets in terms of F1-Score. As can be observed from the table, in regards to F1-Score, E-GraphSAGE outperforms the best reported classifiers in ToN-IoT and BoT-IoT. The NF-ToN-IoT and NF-BoT-IoT experiments achieved an F1-score of 1.0 and 0.97, respectively, which are the same as the state-of-art algorithms.

These results demonstrate the power of our GNN-based approach for traffic classification and network intrusion detection. In contrast to most related works, where a new ML-based NIDS classifier is evaluated on one, two or a maximum three datasets, we have demonstrated the performance of our E-GraphSAGE approach across four significantly different NIDS benchmark datasets. The consistently solid results demonstrate the robustness of our GNN-based approach and its ability to generalise across different types of network traffic patterns, feature sets, and attack types. We believe this is due to the ability of the GNN architecture to capture the information in the inherent graph structure of the flow-based NIDS datasets.

TABLE III: Performance of binary classification by E-GraphSAGE compared with the state-of-art algorithms

Method	Dataset	F1
Proposed E-GraphSAGE	BoT-IoT	**1.00**
XGBoost [5]	BoT-IoT	0.99
Proposed E-GraphSAGE	NF-BoT-IoT	**0.97**
Extra Tree Classifier [6]	NF-BoT-IoT	0.97
Proposed E-GraphSAGE	ToN-IoT	**0.99**
Ensemble [7]	ToN-IoT	0.95
Proposed E-GraphSAGE	NF-ToN-IoT	**1.00**
Extra Tree Classifier [6]	NF-ToN-IoT	1.00

TABLE IV: Results of multiclass classification by E-GraphSAGE on BoT-IoT dataset families

	BoT-IoT		NF-BoT-IoT	
Class Name	**DR**	**F1-Score**	**DR**	**F1-Score**
Benign	100.00%	0.99	99.45%	0.42
DDoS	99.99%	1.00	40.82%	0.39
DoS	99.99%	1.00	57.13%	0.47
Reconnaissance	99.98%	1.00	84.50%	0.92
Theft	93.75%	0.97	99.83%	0.39
Weighted Average	**99.99%**	**1.00**	**78.16%**	**0.81**

B. Multiclass Classification Results

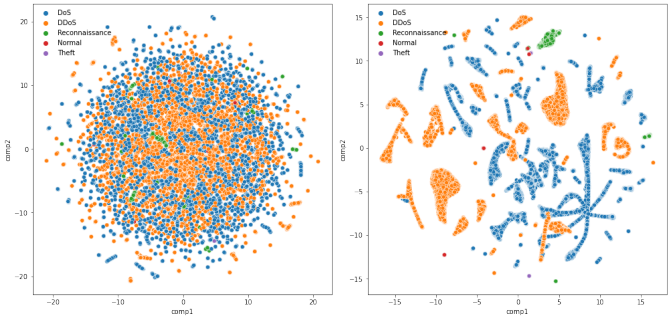
We are now considering the multiclass classification problem, where the classifier aims to distinguish between different types of attacks as well as benign traffic. This is a much harder problem than in the binary case. For the evaluation of the E-GraphSAGE classifier in the multiclass scenario, we considered the same four NIDS datasets as for the binary case. Depending on the NIDS dataset, the number of attack classes range between 4 and 9.

Tables IV and V show the corresponding results for the BoT-IoT and ToN-IoT datasets and their respective Netflow counterparts. For the BoT-IoT dataset with its original format and feature set, a very high weighted Detection Rate and F1-Score is achieved for all the 5 traffic classes (4 attack classes plus benign class), with a weighted average of 99.99% and 1.00 respectively. The corresponding numbers for the NF-BoT-IoT dataset are significantly lower, with a weighted average DR value of 78.16% and weighted F1-Score of 0.81.

The results for the ToN-IoT datasets show a high weighted average Detection Rate (86.78%) and F1-Score (0.87), but the values vary significantly for the individual traffic classes.

TABLE V: Results of multiclass classification by E-GraphSAGE on ToN-IoT datasets families

	ToN-IoT		NF-ToN-IoT	
Class Name	**DR**	**F1-Score**	**DR**	**F1-Score**
Benign	88.12%	0.91	98.86%	0.92
Backdoor	5.06%	0.08	98.38%	0.99
DDoS	96.94%	0.98	52.35%	0.68
DoS	96.08%	0.73	0.00%	0.00
Injection	88.94%	0.83	93.15%	0.71
MIMT	87.43%	0.18	22.88%	0.28
Ransomware	98.55%	0.94	96.49%	0.23
Password	89.15%	0.91	19.92%	0.25
Scanning	75.84%	0.85	15.32%	0.13
XSS	92.08%	0.95	0.00%	0.00
Weighted Average	**86.78%**	**0.87**	**67.16%**	**0.63**



(a) raw data **(b)** edge embedded data

Fig. 4: Visualisation of dimensionality reduction a) Sample of BoT-IoT raw validation data, b)Sample of edge embeddings generated by E-GraphSAGE (Multiclass).

As for the BoT-IoT dataset, we observe a significantly lower classification performance for the NetFlow variant of the BoT-IoT dataset. This can be explained by the very different nature of the feature set. While the original feature sets include specifically engineered features to detect the attack classes included in the dataset, the NetFlow variants consist of relatively simple and generic features.

To get an intuitive understanding of the promising and robust performance of the E-GraphSAGE classifier, we investigated and visualised the edge embeddings generated by the algorithm, based on the BoT-IoT dataset. For this, we generate two visualisations. First, we take the sample 'raw' BoT-IoT dataset and the edge embedding then applied the *Uniform Manifold Approximation and Projection (UMAP)* [23] dimensionality reduction algorithm to map the high-dimensional data down to two dimensions, for the purpose of visualisation. The result is shown in Figure 4. We now see a clear separation of attack flows and normal/benign flows. This illustrates the ability of the edge embedding mechanism of the E-GraphSAGE algorithm to leverage the inherent graph structure of the flow-based NIDS data, and to be able to separate attack traffic from normal traffic in the embedding space, which in turn results in a high classification performance.

As in the binary classification case, we have provided a comparison of our E-GraphSAGE-based NIDS with the state-of-the-art classifiers. Table VI shows the average multiclass

TABLE VI: Performance of multiclass classification by E-GraphSAGE compared with the state-of-art algorithms

Method	Dataset	W-F1
Proposed E-GraphSAGE	BoT-IoT	**1.00**
KNN [8]	BoT-IoT	0.99
XGBoost [5]	BoT-IoT	0.97
Proposed E-GraphSAGE	NF-BoT-IoT	**0.81**
Extra Tree Classifier [6]	NF-BoT-IoT	0.77
Proposed E-GraphSAGE	ToN-IoT	**0.87**
Extra Tree Classifier [6]	ToN-IoT	0.87
Proposed E-GraphSAGE	NF-ToN-IoT	**0.63**
Extra Tree Classifier [6]	NF-ToN-IoT	0.60

weighted F1-Scores of E-GraphSAGE compared with the top one or two best reported results in the literature, depending on availability. We observe that E-GraphSAGE outperforms all the state-of-the-art classifiers, with the only exception of the ToN-IoT dataset, where E-GraphSAGE matches the F1-Score as the best reported results.

Overall, we see that E-GraphSAGE at least matches, and in most cases significantly outperforms the state-of-the-art ML-based NIDS approaches, for both binary and multi-class classification. This is particularly significant, given our stringent comparison methodology, where we picked the best performing classifiers for each of the four individual NIDS datasets, and performed a pairwise comparison with E-GraphSAGE. This is in contrast to a more typical approach in which a classifier is compared with related approaches on one, two, or three common datasets.

VII. Conclusions and Future Work

This paper presents a novel approach to network intrusion detection based on GNNs. For this, we propose E-GraphSAGE which the capturing of edge features as well as the topological pattern of a network flow graph, and is hence able to implement attack flow detection. In this paper, we focus on the application of E-GraphSAGE for IoT network intrusion detection, and more specifically, for the detection of malicious network flows. To the best of our knowledge, this represents the first implementation and extensive evaluation of an GNN-based NIDS for IoT using network flow data. Our experimental evaluation based on four IoT NIDS benchmark datasets shows that our E-GraphSAGE-based NIDS performs exceptionally well and overall outperforms the state-of-the-art ML-based classifiers. The evaluation results of our initial system demonstrate the potential of a GNN-based approach for network intrusion detection. In the future, we plan to apply neighbourhood sampling techniques to improve the run-time of the proposed E-GraphSAGE model, particularly exploring non-uniform sampling techniques. Also, it is worth investigating explainable graph neural network algorithms, such as GNNExplainer [24], to get more insights about GNN model outputs.

References

[1] A. Ghasempour, "Internet of things in smart grid: Architecture, applications, services, key technologies, and challenges," *Inventions*, vol. 4, no. 1, p. 22, 2019.

[2] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021. DOI: 10.1109/TNNLS.2020.2978386.

[3] B. Claise, "Cisco systems netflow services export version 9," *RFC*, vol. 3954, pp. 1–33, 2004.

[4] H. He, X. Sun, H. He, G. Zhao, L. He, and J. Ren, "A novel multimodal-sequential approach based on multi-view features for network intrusion detection," *IEEE Access*, vol. 7, pp. 183 207–183 221, 2019. DOI: 10.1109/ACCESS.2019.2959131.

[5] M. A. Lawal, R. A. Shaikh, and S. R. Hassan, "An anomaly mitigation framework for iot using fog computing," *Electronics*, vol. 9, no. 10, 2020, ISSN: 2079-9292. DOI: 10.3390/electronics9101565.

[6] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, *Netflow datasets for machine learning-based network intrusion detection systems*, Nov. 2020. arXiv: 2011.09144.

[7] P. Kumar, G. P. Gupta, and R. Tripathi, "An ensemble learning and fog-cloud architecture-driven cyber-attack detection framework for iomt networks," *Computer Communications*, vol. 166, pp. 110–124, 2021, ISSN: 0140-3664. DOI: https://doi.org/10.1016/j.comcom.2020.12.003. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0140366420320090.

[8] A. Churcher, R. Ullah, J. Ahmad, S. ur Rehman, F. Masood, M. Gogate, F. Alqahtani, B. Nour, and W. J. Buchanan, "An experimental analysis of attack classification using machine learning in iot networks," *Sensors*, vol. 21, no. 2, 2021, ISSN: 1424-8220. DOI: 10.3390/s21020446. [Online]. Available: https://www.mdpi.com/1424-8220/21/2/446.

[9] G. Vormayr, T. Zseby, and J. Fabini, "Botnet communication patterns," *IEEE Communications Surveys Tutorials*, vol. 19, no. 4, pp. 2768–2796, 2017. DOI: 10.1109/COMST.2017.2749442.

[10] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Surveying port scans and their detection methodologies," *The Computer Journal*, vol. 54, no. 10, pp. 1565–1581, 2011.

[11] G. Kambourakis, T. Moschos, D. Geneiatakis, and S. Gritzalis, "Detecting dns amplification attacks," in *International workshop on critical information infrastructures security*, Springer, 2007, pp. 185–196.

[12] Q. Xiao, J. Liu, Q. Wang, Z. Jiang, X. Wang, and Y. Yao, "Towards Network Anomaly Detection Using Graph Embedding," in *Computational Science – ICCS 2020*, V. V. Krzhizhanovskaya, G. Závodszky, M. H. Lees, J. J. Dongarra, P. M. A. Sloot, S. Brissos, and J. Teixeira, Eds., Cham: Springer International Publishing, 2020, pp. 156–169, ISBN: 978-3-030-50423-6.

[13] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, pp. 1025–1035, ISBN: 9781510860964.

[14] J. Zhou, Z. Xu, A. Rush, and M. Yu, "Automating Botnet Detection with Graph Neural Networks," in *4th Workshop on Machine Learning and Systems (MLSys)*, 2020.

[15] L. Gong and Q. Cheng, "Exploiting edge features for graph neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2019.

[16] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and Y. W. Teh, Eds., ser. Proceedings of Machine Learning Research, vol. 70, PMLR, Jun. 2017, pp. 1263–1272. [Online]. Available: https://proceedings.mlr.press/v70/gilmer17a.html.

[17] H. Cai, V. W. Zheng, and K. C. Chang, "A Comprehensive Survey of Graph Embedding: Problems, Techniques, and Applications," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 9, pp. 1616–1637, 2018, ISSN: 1558-2191. DOI: 10.1109/TKDE.2018.2807452.

[18] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, and M. Sun, "Graph neural networks: A review of methods and applications," *ArXiv*, vol. abs/1812.08434, 2018.

[19] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, 2019, ISSN: 0167739X. DOI: 10.1016/j.future.2019.05.041. arXiv: 1811.00701.

[20] A. Alsaedi, N. Moustafa, A. M. Z. Tari, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165 130–165 150, 2020. DOI: 10.1109/ACCESS.2020.3022862.

[21] *Low-code programming for event-driven applications*, Feb. 2021. [Online]. Available: https://nodered.org/.

[22] *An extensible netflow v5/v9/ipfix probe for ipv4/v6*, Feb. 2021. [Online]. Available: https://www.ntop.org/products/netflow/nprobe/.

[23] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," *arXiv preprint arXiv:1802.03426*, 2018.

[24] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "Gnnexplainer: Generating explanations for graph neural networks," *Advances in neural information processing systems*, vol. 32, p. 9240, 2019.